

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики – процессов управления

Шаго Павел Евгеньевич

Выпускная квалификационная работа

**«Математическое моделирование и реализация
планировщика процессов для гетерогенных
процессорных архитектур»**

Уровень образования: бакалавриат

Направление: 01.03.02 Прикладная математика и информатика

Основная образовательная программа СВ.5005.2022: «Прикладная математика,
фундаментальная информатика и программирование»

Научный руководитель
кандидат физ.-мат. наук, доцент
Корхов Владимир Владиславович

Рецензент
Эксперт по архитектуре и сетевому стеку
ООО НИЦ АЭРОДИСК
Анохин Юрий Владимирович

Санкт-Петербург
2026

Содержание

Введение	3
Постановка задачи	4
1 Подходы к реализации планировщиков в ОС	5
1.1 Задача планирования	5
2 Математические модели планировщиков	8
3 Реализация алгоритмов планирования	9
4 Описание тестового окружения и экспериментального оборудования	10
5 Анализ результатов эксперимента	11
Заключение	12
Список литературы	13

Введение

Постановка задачи

Объектом исследования являются алгоритмы планирования процессов в операционных системах на вычислительных устройствах с гетерогенной процессорной архитектурой (Intel Hybrid CPU, ARM big.LITTLE).

Целью работы является разработка и экспериментальная оценка планировщика, обеспечивающего более высокую эффективность вычислений на гетерогенном оборудовании по сравнению с базовым алгоритмом EEVDF ядра Linux.

Для достижения цели работы необходимо:

1. Проанализировать и определить наиболее перспективный подход к реализации планировщика.
2. Построить математическую модель, учитывающую различную вычислительную мощность ядер процессора.
3. Реализовать предложенный алгоритм с использованием инфраструктуры `sched_ext` и разработать воспроизводимый набор бенчмарков.
4. Провести эксперимент на гетерогенном оборудовании и оценить результаты в сравнении с базовым EEVDF.

1 Подходы к реализации планировщиков в ОС

1.1 Задача планирования

В современных операционных системах механизмы многозадачности являются фундаментальным инструментом управления вычислительными ресурсами. Реализация концепции псевдопараллелизма базируется на процедурах контекстного переключения, которые позволяют распределять кванты процессорного времени между потоками исполнения.

Ключевым вызовом в данном контексте является задача построения эффективного расписания, при котором порядок выполнения процессов минимизирует простой оборудования и максимизирует целевые показатели эффективности.

В книге [1] систематизированы алгоритмы планирования: от классических FIFO, Round-Robin и приоритетных схем до моделей пропорционального разделения ресурсов. Авторы формализуют ключевые метрики, время ожидания, время оборота и утилизацию ресурсов, и показывают, что оптимальность по одной метрике достигается, как правило, ценой деградации другой. Работа остаётся фундаментальным справочником по теории планирования.

В учебнике [7] описаны практические подходы к реализации планировщиков в операционных системах, включая устройство многоуровневых очередей, механизмы вытеснения и балансировки нагрузки между ядрами. В частности, рассмотрены принципы работы CFS и его преемника EEVDF в ядре Linux, а также ограничения, с которыми сталкивается любой монолитный планировщик при попытке одновременно удовлетворить разнородные нагрузки. Изменение алгоритма требует перекомпиляции ядра и перезагрузки машины, а ошибка в коде планировщика с высокой вероятностью приводит к отказу системы, что делает разработку внутри ядра дорогим процессом.

В работе [5] Stoica и Abdel-Wahab предложили алгоритм Earliest Eligible Virtual Deadline First как механизм пропорционального разделения ресурсов с явными гарантиями задержки. Каждой задаче назначается виртуальное допустимое время VE и виртуальный дедлайн $VD = VE + r/w$, где r составляет объём запрошенного CPU-времени, а w обозначает вес задачи. Глобальное виртуальное время растёт обратно пропорционально суммарному весу активных задач, что гарантирует каждой задаче долю процессора пропорционально её весу. Планировщик выбирает задачу с наименьшим VD среди допустимых, то есть тех, чья VE не превышает текущего виртуального времени. Это одновременно исключает голодание задач с малым весом и несправедливое вытеснение.

В ядре Linux 6.6 EEVDF заменил Completely Fair Scheduler, устранив отсутствие явного механизма задания требований к задержке и непредсказуемость поведения при высокой нагрузке [9].

В книге [8] Rice детально описана технология eBPF, лежащая в основе фреймворка `sched_ext`. Рассмотрен полный путь от написания eBPF-программы до её верификации и JIT-компиляции в ядре. Верификатор статически проверяет завершенность кода и безопасность операций с памятью ещё до исполнения, что делает eBPF принципиально более безопасным инструментом расширения ядра по сравнению с загружаемыми модулями.

В работе [12] Humphries и соавторы из Google представили фреймворк ghOST (Google Host OS Threads scheduler), делегирующий решения о планировании агенту в пространстве пользователя. Ядро передаёт события планирования агенту через разделяемую память без копирования данных, агент отвечает управляющими командами. Такая схема открывает разработчику произвольные структуры данных, стандартные профилировщики и быструю итерацию без перезагрузок. Google применила ghOST во внутренних дата-центрах и показала, что специализированный агент способен обеспечивать более низкую задержку, чем штатный планировщик ядра. Фреймворк требует отдельного патча ядра, а обращение к пространству пользователя на критическом пути вносит дополнительную латентность, ограничивая применимость фреймворка средами, где команда контролирует и ядро, и прикладное окружение.

В репозитории проекта `sched_ext` [10] собрана коллекция планировщиков, реализующих разные стратегии на одной платформе. Сам фреймворк, принятый в основную ветку Linux 6.12 в 2024 году, загружает планировщик как набор eBPF-программ через стандартный системный вызов `bpf()` без перекомпиляции и перезагрузки [11]. Среди реализаций выделяется `scx_simple`, минималистичный планировщик на взвешенном виртуальном времени без вытеснения, полезный как воспроизводимый `baseline`, и `Rusty`, написанный на языке Rust планировщик с акцентом на балансировку нагрузки с учётом топологии процессора.

В работе [2] Galin и Hedblom провели сравнительную оценку планировщиков LAVD, `scx_simple` и EEVDF применительно к телекоммуникационным нагрузкам в среде Kubernetes. LAVD (Latency-criticality Aware Virtual Deadline) назначает виртуальные дедлайны с учётом критичности задачи по частоте пробуждений, доле времени ожидания и объёму потребленного CPU. При высокой нагрузке LAVD сократил среднее время оборота задач на 6–81% и улучшил хвостовую задержку (99-й перцентиль) до 90% относительно EEVDF. Примечательно, что при умеренной нагрузке `scx_simple` стабильно превосходил LAVD, что свидетельствует о том, что преимущество принадлежит не конкретному алгоритму, а гибкости самой инфраструктуры `sched_ext`.

В работе [3] Мао, Ujсich и Ма представили планировщик Aegis, разработанный для защиты систем аудита безопасности от угрозы суперпродюсера. Процесс с аномально высокой частотой генерации событий способен переполнить буфер сборщика, вынуждая его отбрасывать критически важные записи и тем самым скрывая следы атаки. Aegis реализован поверх sched_ext с применением обучения с подкреплением (DeepQ-Network) непосредственно внутри eBPF-программы. Авторы зафиксировали, что EEVDF, LAVD и Rusty теряют от 39% до более чем 98% системных событий при высокой нагрузке, тогда как Aegis обеспечил нулевую потерю во всех тестовых сценариях при накладных расходах менее 0.5% в типичных условиях.

В работе [4] Wang и соавторы предложили систему Mixture-of-Schedulers, строящую интеллектуальный маршрутизатор поверх портфеля специализированных sched_ext-планировщиков. Агент непрерывно наблюдает за CPU, вводом/выводом и сетевым трафиком через eBPF и procfs, XGBoost-классификатор распознаёт тип нагрузки, и sched_ext переключает активный планировщик в реальном времени. Система превзошла EEVDF в 86.4% тестовых сценариев при накладных расходах около 1.45% одного ядра процессора, а также показала хорошую обобщаемость при переносе с архитектуры x86_64 на ARM. Подход демонстрирует, что рост качества планирования достижим не только через усложнение одного алгоритма, но и через разумную специализацию нескольких простых политик.

2 Математические модели планировщиков

3 Реализация алгоритмов планирования

4 Описание тестового окружения и экспериментального оборудования

5 Анализ результатов эксперимента

Заключение

Список литературы

- [1] Joseph Y-T. Leung Handbook of Scheduling: Algorithms, Models, and Performance Analysis CRC Press 2004
- [2] Michelle Galin, Anna Hedblom Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads Linköping University 2025
- [3] Jinsong Mao, Benjamin E. Ujcich, Shiqing Ma Rethinking Provenance Completeness with a Learning-Based Linux Scheduler arXiv 2025
- [4] Xinbo Wang, Shian Jia, Ziyang Huang, Jing Cao, Mingli Song Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies arXiv 2025
- [5] Ion Stoica, Hussein Abdel-Wahab Earliest Eligible Virtual Deadline First : A Flexible and Accurate Mechanism for Proportional Share Resource Allocation Old Dominion University 1996
- [6] A1349 Pavel Shago [Электронный ресурс]. – Режим доступа: <https://github.com/shgpavel/A1349> – Дата обращения 16.12.2025.
- [7] Andrew S. Tanenbaum, Herbert Bos MODERN OPERATING SYSTEMS Pearson Education 2023
- [8] Liz Rice Learning eBPF O'Reilly Media, Inc 2023
- [9] Torvalds L. Linux Kernel [Электронный ресурс] / L. Torvalds. – Режим доступа: <https://github.com/torvalds/linux> – Дата обращения 04.12.2025.
- [10] sched-ext/scx [Электронный ресурс]. – Режим доступа: <https://github.com/sched-ext/scx> – Дата обращения 07.12.2025.
- [11] lkml [Электронный ресурс]. – Режим доступа: <https://lkml.org> – Дата обращения 24.11.2025.
- [12] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, Christoph Lameter ghOST: Scheduling Beyond the OS ACM SIGOPS 2021